

# DevOps Training Program

## Challenge Lab Workbook

*Advanced & Tricky Tasks — EC2 · IAM · Linux · Jenkins*

### **Lab 7: EC2, IAM & Linux Challenges (6 Tasks)**

### **Lab 8: Jenkins Challenges (6 Tasks)**

These tasks involve debugging, misconfigurations, and real-world edge cases — think before you type!

## LAB 7 • CHALLENGE EDITION

# EC2, IAM & Linux — Challenge Tasks



Duration: 3–4 Hours



Difficulty: Advanced ★★ ★

### HEADS UP

These tasks are intentionally tricky — some involve broken configurations you must fix, others test edge cases that trip up real engineers. Read each scenario carefully before acting.

## TASK 1: The Locked-Out Instance

### Scenario / Statement

You launched an EC2 instance but lost your private key (.pem file). The instance is running and you cannot SSH into it. Your task: recover access to this instance WITHOUT terminating it or losing its data.

### Hints (don't peek unless stuck!)

- EBS volume detach + reattach to a 'rescue' instance approach
- You can edit `authorized_keys` by mounting the root volume on another instance
- AWS Systems Manager (SSM) Session Manager is a key-less alternative if the SSM agent is running

✓ **Expected Outcome:** Student regains shell access to the instance using either the volume-swap method or SSM Session Manager, and documents which method worked and why.

## TASK 2: Security Group Lockout Puzzle

### Scenario / Statement

A teammate configured a Security Group with an inbound rule allowing SSH (port 22) only from a specific IP — but they typed the wrong IP and now NO ONE can connect, including you. The instance must remain reachable by SSH within 10 minutes for a 'production fix'.

### Hints (don't peek unless stuck!)

- Security Group changes apply instantly — no instance restart needed
- Find your current public IP using a quick check (e.g., `curl ifconfig.me`) before editing the rule
- Differentiate between Security Group (stateful, instance-level) and NACL (stateless, subnet-level) — check both!

✓ **Expected Outcome:** Student identifies the incorrect SG rule, corrects the source IP/CIDR, and successfully re-establishes SSH access — with a screenshot of the before/after rule.

## TASK 3: The IAM Permission Trap

### Scenario / Statement

You create an IAM user with an attached policy that **ALLOWS** `s3:GetObject` on a bucket, but the user still gets 'Access Denied' when trying to download a file. There is no explicit Deny anywhere in the user's own policies. Find out why.

### Hints (don't peek unless stuck!)

- Check the S3 bucket policy itself — not just the IAM user policy
- Check if the bucket has 'Block Public Access' or an explicit Deny statement
- Remember: IAM policy AND resource-based policy both need to allow the action — an explicit Deny anywhere wins

✓ **Expected Outcome:** Student identifies a conflicting bucket policy or Block Public Access setting causing the implicit/explicit deny, fixes it, and successfully downloads the object.

## TASK 4: Disk Full, Service Down

### Scenario / Statement

Your Jenkins EC2 instance's root volume (/) is at 100% disk usage and Jenkins has stopped responding. You cannot resize the volume (assume it's the max allowed for this exercise). Find and clean up space WITHOUT deleting anything Jenkins needs to function.

### Hints (don't peek unless stuck!)

- Use `df -h` and `du -sh /*` to locate the largest space consumers
- Check `/var/log` for oversized log files and `/var/lib/jenkins/workspace` for old build artifacts
- `journalctl` can also consume huge space — check `journalctl --disk-usage` and `vacuum` if needed

✓ **Expected Outcome:** Student frees up sufficient disk space (document before/after `df -h` output) and confirms Jenkins is reachable again on port 8080.

## TASK 5: Cross-Account Mystery Role

### Scenario / Statement

Your manager asks you to set up an IAM Role that allows a DIFFERENT AWS account (a partner company) to access a specific S3 bucket in YOUR account — but only to read objects, and only for the next 24 hours of testing. Design and implement this securely.

### Hints (don't peek unless stuck!)

- This requires a Role with a Trust Policy specifying the external Account ID as principal
- Use an IAM condition with `aws:CurrentTime` or a scheduled cleanup — or simply document the manual revoke step
- Cross-account access never needs a shared password or access key — only Role ARNs and Trust Policies

✓ **Expected Outcome:** Student creates a cross-account IAM role with a correctly scoped trust policy and a read-only permissions policy, and explains how/when they'd revoke it.

## TASK 6: Zombie Process & Port Conflict

### Scenario / Statement

Jenkins won't start on port 8080 — it throws 'Address already in use'. Find what's holding the port hostage, resolve the conflict, and get Jenkins running again WITHOUT changing Jenkins' default port.

### Hints (don't peek unless stuck!)

- `lsof -i :8080` or `netstat -tlnp | grep 8080` shows the PID holding the port
- Sometimes a previous Jenkins process didn't shut down cleanly (zombie/orphaned process)
- `kill` vs `kill -9` — try graceful shutdown first before force-killing

✓ **Expected Outcome:** Student identifies the conflicting process by PID, terminates it cleanly, and starts Jenkins successfully on port 8080 — with proof via curl localhost:8080.

## LAB 8 • CHALLENGE EDITION

# Jenkins — Challenge Tasks



Duration: 3–4 Hours



Difficulty: Advanced ★★ ★

### HEADS UP

These Jenkins tasks simulate real debugging scenarios — broken pipelines, credential issues, and plugin conflicts. Take screenshots of errors **BEFORE** you fix them, for comparison.

## TASK 1: The Pipeline That Won't Checkout

### Scenario / Statement

A Jenkins Pipeline job fails at the 'Checkout' stage with a Git authentication error, even though the GitHub repo is public. Diagnose why Jenkins can't pull the repository and fix it.

### Hints (don't peek unless stuck!)

- Public repos still sometimes fail due to a misconfigured Jenkins Git plugin or wrong branch name (main vs master)
- Check Manage Jenkins → Global Tool Configuration for Git executable path issues
- Console Output usually shows the EXACT git command Jenkins ran — replicate it manually via SSH to debug

✓ **Expected Outcome:** Student identifies the root cause (e.g., wrong branch, malformed repo URL, or missing Git binary) and gets the Checkout stage to pass.

## TASK 2: Credential Injection Without Leaking Secrets

### Scenario / Statement

You need your Jenkins Pipeline to SSH into another EC2 instance and run a command — but the private key must NEVER appear in Console Output or be hardcoded in the Jenkinsfile. Implement this securely.

### Hints (don't peek unless stuck!)

- Use Jenkins Credentials Store (Manage Jenkins → Credentials) to store the SSH key as 'SSH Username with private key'
- Use the sshagent or withCredentials block inside the Jenkinsfile to inject it temporarily
- Jenkins automatically masks known credential values in Console Output — verify this by checking the log

✓ **Expected Outcome:** Student stores the key securely in Jenkins Credentials, references it via withCredentials/sshagent in the Jenkinsfile, and confirms the Console Output shows '\*\*\*\*\*' instead of the real key.

## TASK 3: The Silent Failure

### Scenario / Statement

A pipeline stage runs a shell script that actually fails internally (e.g., a typo'd command), but Jenkins marks the BUILD as SUCCESS anyway. Find out why Jenkins isn't catching the failure, and fix the pipeline so it properly fails when a command errors out.

**Hints (don't peek unless stuck!)**

- By default, only the LAST command's exit code in a multi-line sh step determines stage success
- Add set -e at the top of multi-line shell blocks to fail-fast on any error
- Chained commands with && vs ; behave very differently for error propagation

✓ **Expected Outcome:** Student modifies the Jenkinsfile shell step to properly propagate failures (using set -e or correct exit code handling) and demonstrates a deliberately broken command now correctly fails the build.

## TASK 4: Two Jobs, One Race Condition

**Scenario / Statement**

You have two Jenkins jobs that both deploy to the same EC2 instance using the same deploy.sh script. When triggered simultaneously (e.g., two rapid git pushes), they sometimes corrupt each other's deployment. Fix the pipeline so concurrent builds don't collide.

**Hints (don't peek unless stuck!)**

- Jenkins has a built-in 'Throttle Concurrent Builds' option or the simpler option { disableConcurrentBuilds() } directive
- Consider using a lock resource (Lockable Resources plugin) if multiple DIFFERENT jobs share the same target server
- Test by triggering 2 builds within seconds of each other and observing the queue behavior

✓ **Expected Outcome:** Student configures concurrency control (disableConcurrentBuilds or equivalent) and proves via the Build Queue that the second build now waits instead of racing.

## TASK 5: Plugin Conflict Meltdown

**Scenario / Statement**

After installing a new Jenkins plugin, the ENTIRE Jenkins dashboard fails to load (white screen or 500 error) when you restart the service. You cannot access the UI to disable the plugin. Recover Jenkins.

**Hints (don't peek unless stuck!)**

- Plugins live as .jpi/.hpi files in /var/lib/jenkins/plugins/ on the filesystem
- You can rename or delete the problematic plugin file directly via SSH, then restart Jenkins
- Check /var/log/jenkins/jenkins.log for the exact plugin/class causing the startup crash

✓ **Expected Outcome:** Student identifies the broken plugin from the Jenkins log, removes/disables it via the filesystem, restarts the Jenkins service, and confirms the dashboard loads again.

## TASK 6: The Webhook That Cried Wolf

**Scenario / Statement**

Your GitHub webhook is configured correctly and shows 'delivered successfully' (green check) on GitHub's side, but Jenkins never actually starts a new build. Diagnose the disconnect between GitHub and Jenkins.

### Hints (don't peek unless stuck!)

- Check Jenkins job config: is 'GitHub hook trigger for GITScm polling' actually enabled on the job?
- Check if a reverse proxy / firewall / Security Group is silently dropping the incoming webhook POST despite GitHub showing 'success' (GitHub only confirms TCP delivery, not Jenkins processing)
- Manually test using Jenkins' own 'Recent Deliveries' redelivery button while watching `/var/log/jenkins/jenkins.log` live

✓ **Expected Outcome:** Student finds the actual breakpoint (e.g., trigger checkbox unchecked, or payload URL slightly wrong) and demonstrates a live git push successfully auto-triggering a new build.